НИЯУ МИФИ

Модуль “Бинарные уязвимости”

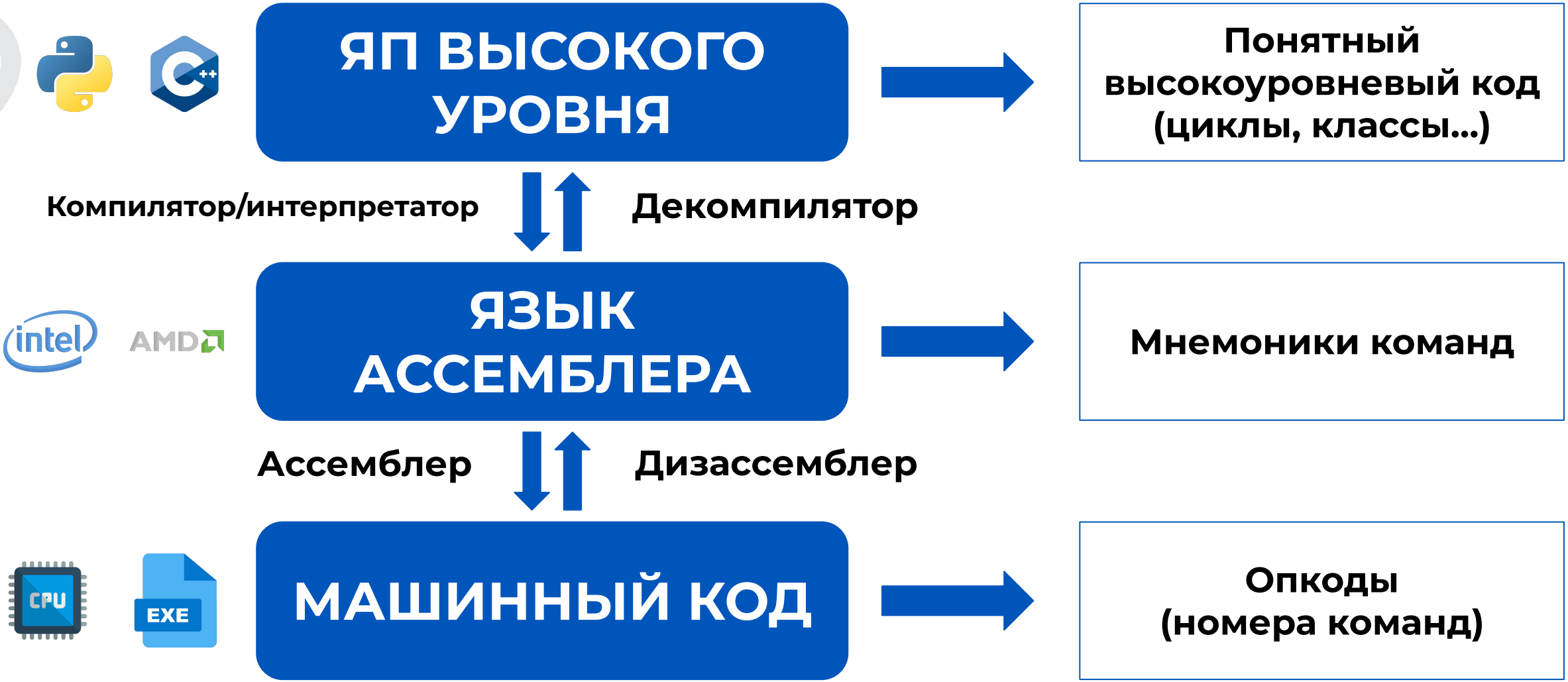
Урок 3. Язык ассемблера. Переполнение стека

План занятия

НИЯУ МИФИ

Язык ассемблера. Переполнение стека

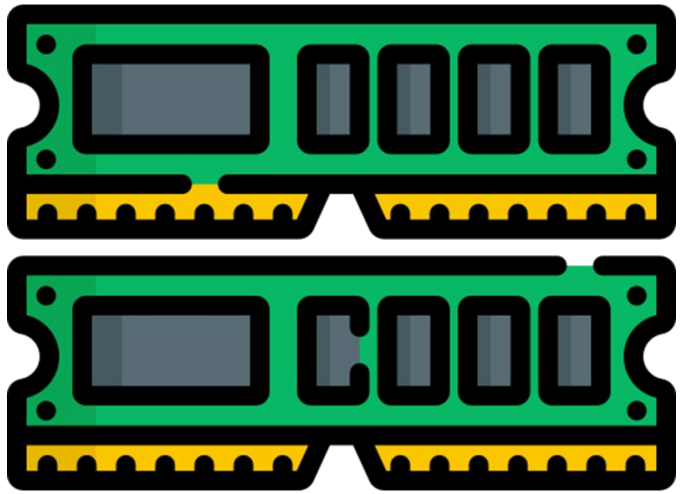
1. Архитектура x86_64
2. Основы языка ассемблера
3. Стек
4. Переполнение стека
5. Решение задач


ЯП ВЫСОКОГО УРОВНЯ
Понятный высокоуровневый код (циклы, классы...)
Компилятор/интерпретатор
Декомпилятор
ЯЗЫК АССЕМБЛЕРА
Мнемоники команд
Ассемблер
Дизассемблер
МАШИННЫЙ КОД
Опкоды (номера команд)
intel
AMD
CPU
EXE

Архитектура x86_64

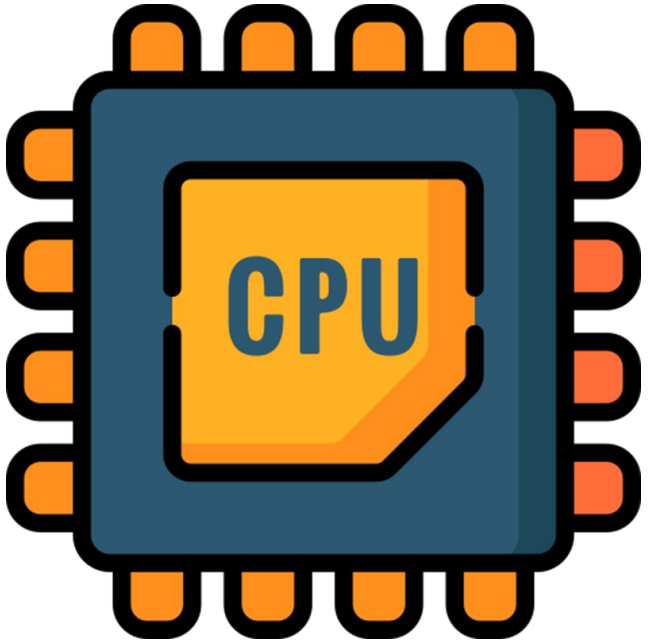
Память

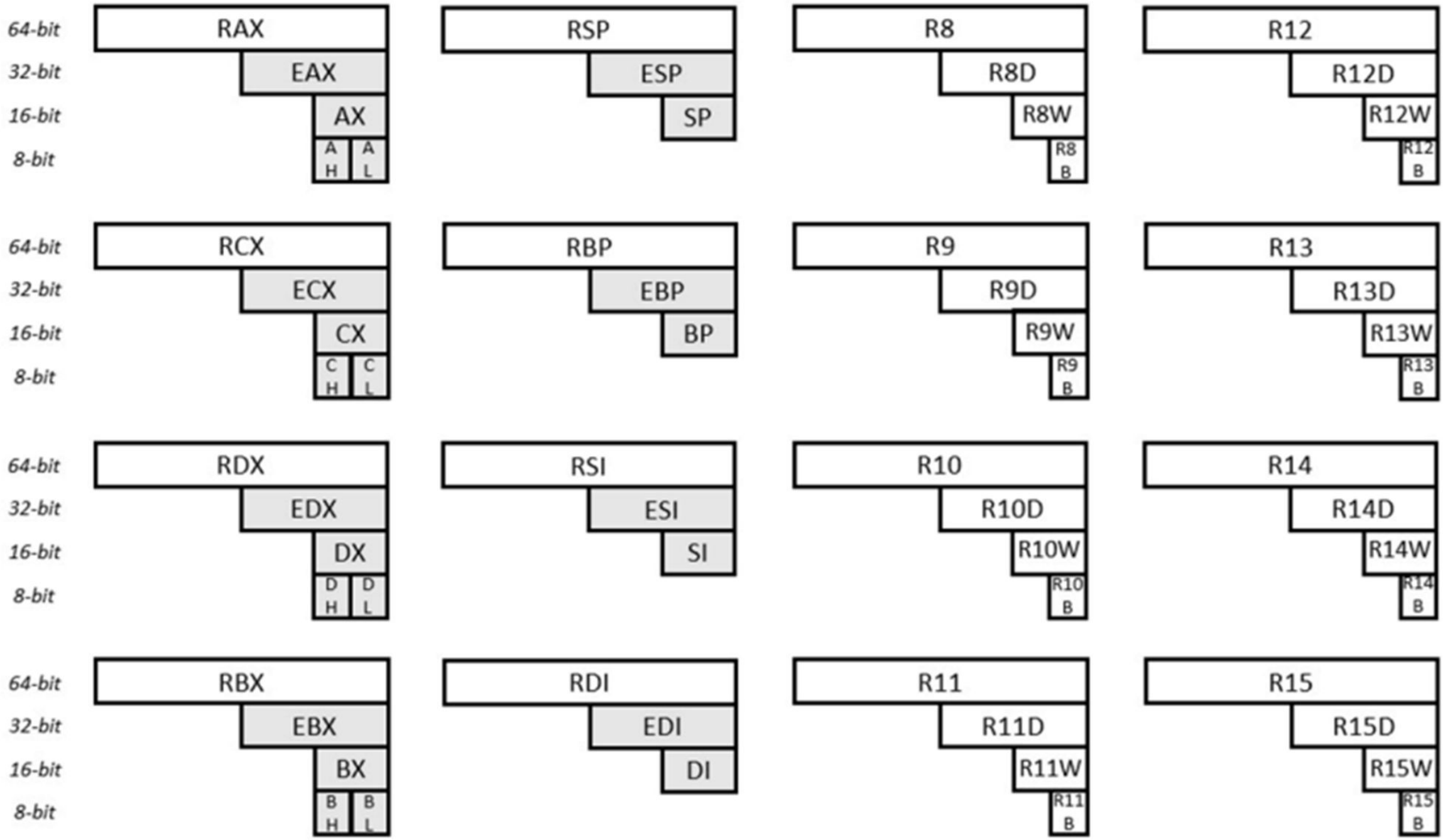
1. Архитектура **64-битная** => адресов 2^64 (фактически 2^48 = 256 ТБ)
2. Регистры -> кэш -> оперативная память -> постоянная память
3. Можем использовать регистры общего назначения (РОН)
4. Регистры 64-битные, но есть доступ к 32, 16 и 8 битам



Регистры общего назначения (РОН)

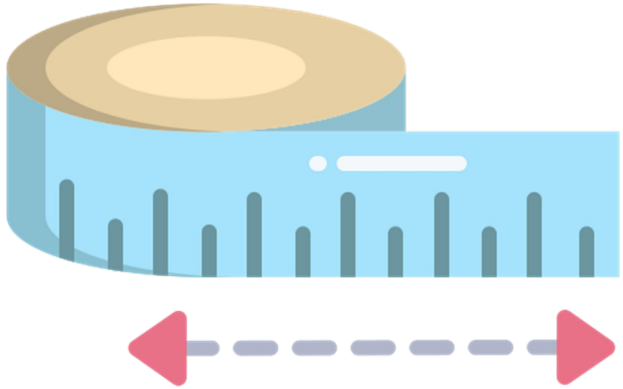
1. **RAX** (accumulator) – для арифметических операций
2. **RBX** (base)
3. **RCX** (counter)
4. **RDX** (data) – арифметика, ввод / вывод
5. **RSP** (stack pointer) – верхушка стека
6. **RBP** (bottom pointer) – низ стека для функции
7. **RDI** (destination index)
8. **RSI** (source index)
9. **RIP** (instruction pointer)
10. **RFLAGS** – флаги состояния




64-bit
32-bit
16-bit
8-bit
RAX
EAX
AX
AH
AL
RSP
ESP
SP
R8
R8D
R8W
R8B
R12
R12D
R12W
R12B
RCX
ECX
CX
CH
CL
RBP
EBP
BP
R9
R9D
R9W
R9B
R13
R13D
R13W
R13B
RDX
EDX
DX
DH
DL
RSI
ESI
SI
R10
R10D
R10W
R10B
R14
R14D
R14W
R14B
RBX
EBX
BX
BH
BL
RDI
EDI
DI
R11
R11D
R11W
R11B
R15
R15D
R15W
R15B

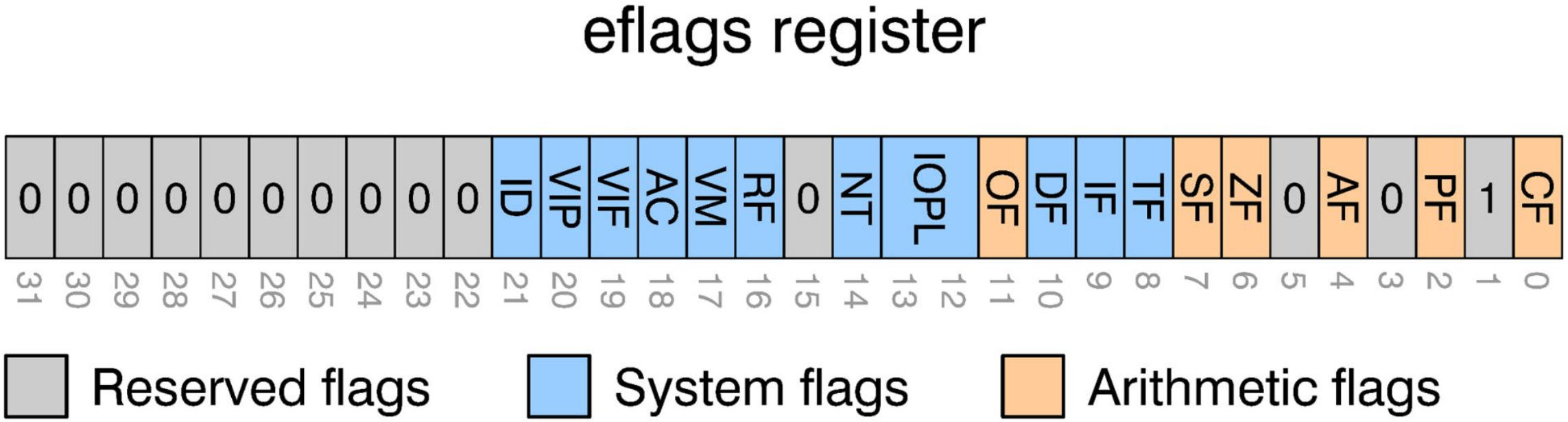
Размеры

1. **Byte** – 1 байт
2. **Word** – 2 байта
3. **Dword** (doubleword) – 4 байта
4. **Qword** (quadword) – 8 байт



RFLAGS

1. **CF** - флаг переноса
2. **ZF** - флаг нуля
3. **OF** - флаг переполнения
4. **DF** - флаг направления



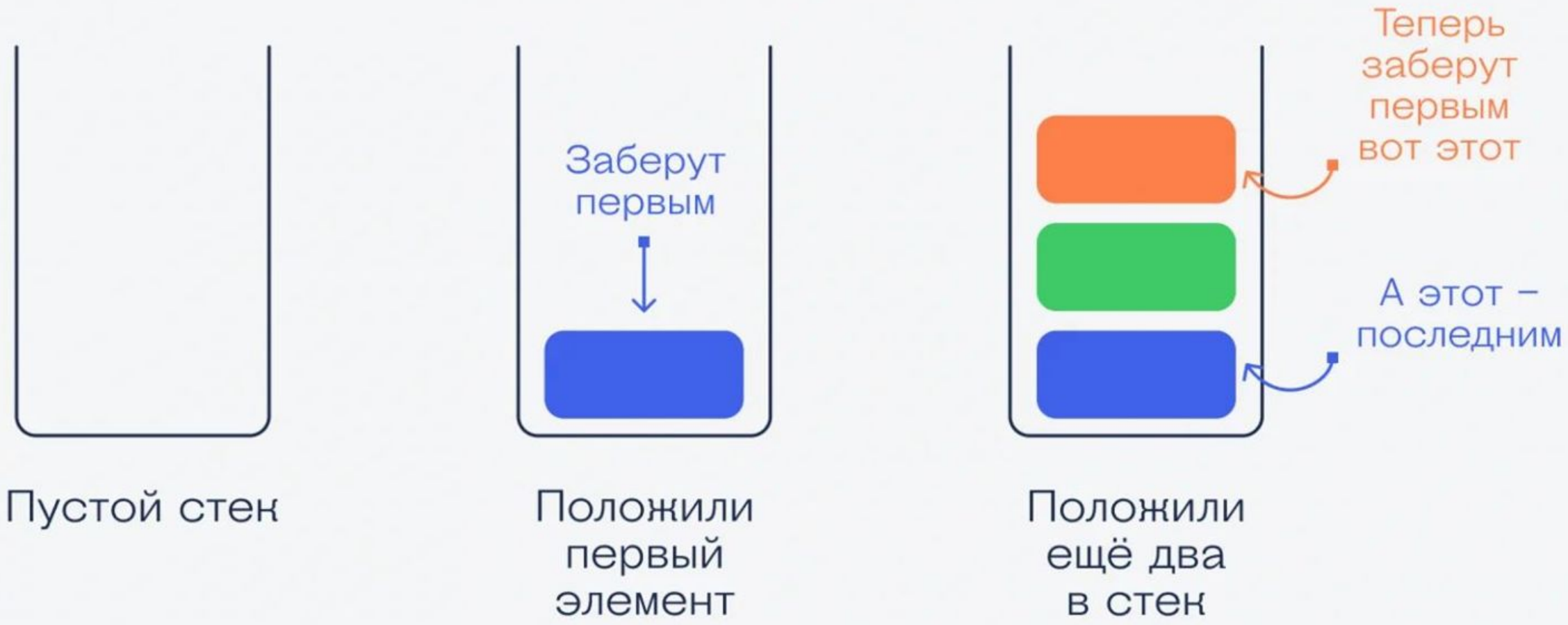
Основы языка ассемблера

Базовые инструкции

1. **mov dst, src** - копирование данных
2. **add a, b** - сложение
3. **sub a, b** - вычитание
4. **cmp a, b** - сравнение (изменяет RFLAGS)
5. **jmp addr** - безусловный переход
6. **je, jne, jg, jl addr** - условный переход на основе значений RFLAGS
7. **nop** - ничего не делать

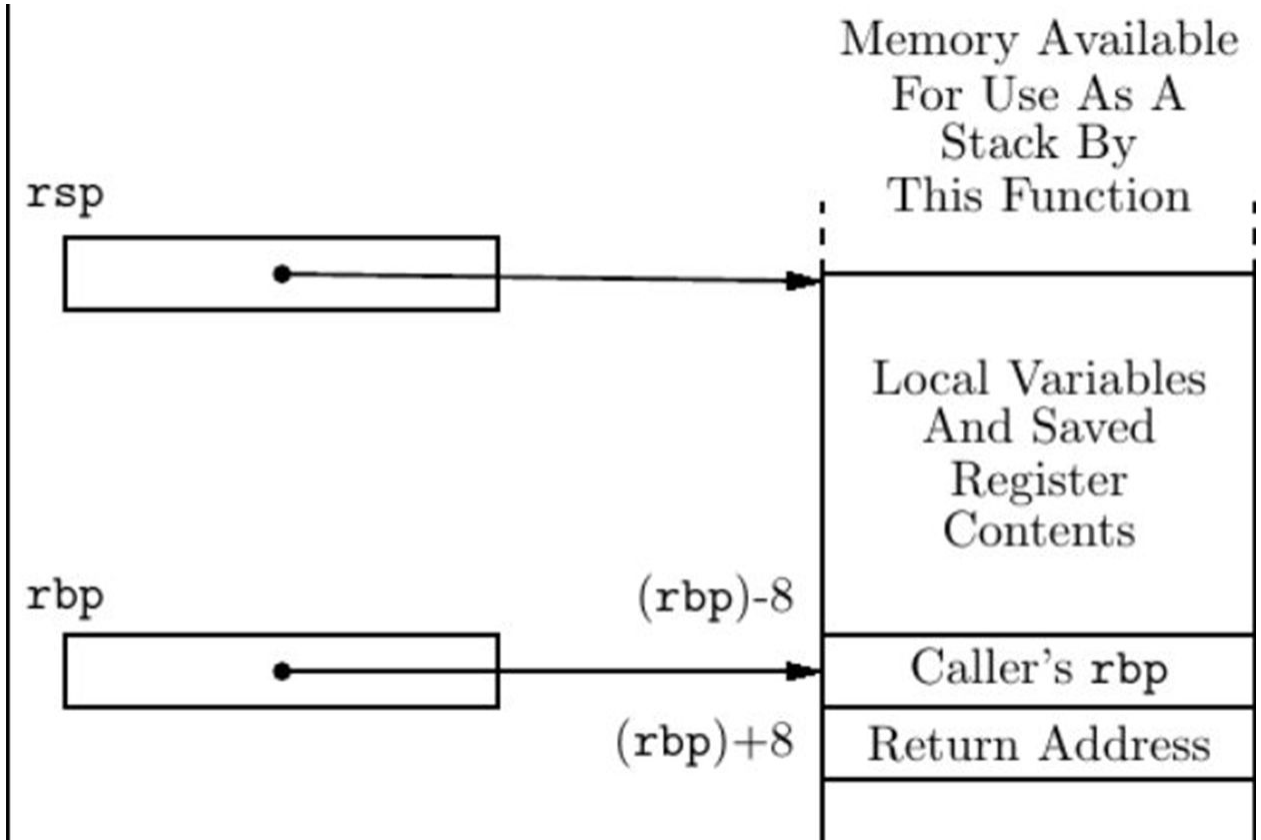
Стек

Стек


Заберут первым
Теперь заберут первым вот этот
А этот – последним
Пустой стек
Положили первый элемент
Положили ещё два в стек

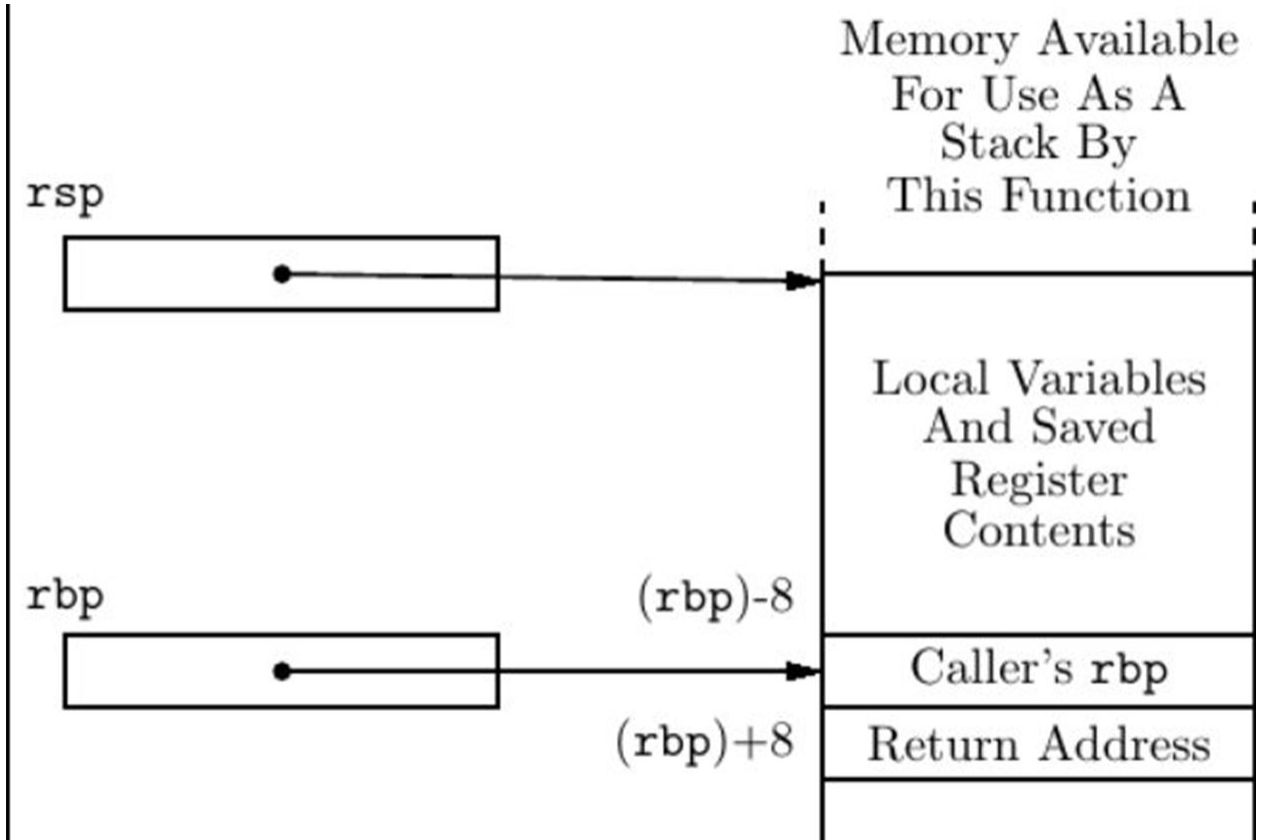
Стек

- **Стек** - область памяти для локальных данных функции
- Растёт от старших адресов к младшим
- RSP - указатель на вершину стека
- RBP - указатель на базу текущего фрейма



Инструкции для работы со стеком

- **push rax** → RSP -= 8; [RSP] = RAX
- **pop rax** → RAX = [RSP]; RSP += 8
- **call func** → push RIP; jmp func (сохраняет адрес возврата)
- **ret** → pop RIP (возвращает управление)



Calling conventions

1. Первые 6 аргументов целого типа: **RDI, RSI, RDX, RCX, R8, R9**
2. Аргументы 7+ передаются через стек
3. Возврат значения: **RAX**
4. Caller-saved: **RAX, RCX, RDX, RSI, RDI, R8–R11** (функция может менять)
5. Callee-saved: **RBX, RBP, R12–R15** (функция должна сохранить, если использует)

Пролог и эпилог

push rbp ; сохранить старый RBP
mov rbp, rsp ; новый RBP = текущий RSP
sub rsp, 32 ; выделить 32 байта под локальные переменные

mov rsp, rbp ; освободить локальные переменные
pop rbp ; восстановить старый RBP
ret ; pop RIP и вернуться

C++ source #1

```cpp
// Type your code here, or load an example.
int square(int num) {
    return num * num;
}
```

x86-64 gcc 14.2 (Editor #1)

x86-64 gcc 14.2

```asm
square(int):
        push    rbp
        mov     rbp, rsp
        mov     DWORD PTR [rbp-4], edi
        mov     eax, DWORD PTR [rbp-4]
        imul    eax, eax
        pop     rbp
        ret
```

Состояние стека

0x30	buffer
0x10	rbp
0x8	ret_addr
0x0	64

Можно перезаписать ret_addr, если нет защиты

C source #1

```c
#include <unistd.h>

void func() {
    char buffer[32];
    read(0, buffer, 128);
}

int main() {
    int a = 64;
    func();
}
```

x86-64 gcc 14.2 (Editor #1)

```asm
func:
        push    rbp
        mov     rbp, rsp
        sub     rsp, 32
        lea     rax, [rbp-32]
        mov     edx, 128
        mov     rsi, rax
        mov     edi, 0
        call    read
        nop
        leave
        ret
main:
        push    rbp
        mov     rbp, rsp
        sub     rsp, 16
        mov     DWORD PTR [rbp-4], 64
        mov     eax, 0
        call    func
        mov     eax, 0
        leave
        ret
```

Решение задач

Митигации

1. **ASLR** (рандомизация базового адреса ELF библиотек)

vmmap
aslr on

2. **PIE** (рандомизация базового адреса самого исполняемого файла)
3. **Full RELRO** (read-only GOT)
4. **Stack Canary** (секретное значение на стеке, которое мешает захватывать RIP)

Полезные материалы

1. pwn.spbctf.ru/tasks
2. pwn.college
3. play.picoctf.org
4. Руководство по ассемблеру NASM

Спасибо
за внимание!